

TradePilot Security Audit Report

- Executive Summary
- Threat 1: Telegram Bot Token Exposed
- Threat 2: User Analytics Database in Version Control
- Threat 3: Admin Dashboard Has No Authentication
- Threat 4: Cross-Origin API Access (CORS Vulnerability)
- Threat 5: Scoring Algorithm Exposed Through API
- Threat 6: ML Model Weights and Architecture Exposed
- Threat 7: Source Code is a Complete Blueprint
- Threat 8: Paper Trade Data Reveals Strategy Details
- Threat 9: No User Authentication System
- Threat 10: Unpinned Software Dependencies
- Threat 11: No Input Validation on API Parameters
- Threat 12: Server Denial of Service (DDoS)
- Threat 13: Network Interception (Man-in-the-Middle)
- Threat 14: Shared In-Memory State
- Summary: Priority Action List
- The Golden Rule

TradePilot Security Audit Report

Project	TradePilot — AI-Powered Trading Platform
Version	v0.3 (Paper Trading Phase)
Audit Date	April 15, 2026
Status	CRITICAL — Immediate action required
Classification	CONFIDENTIAL — Internal Use Only

Prepared By	TradePilot Security Team
For	Soumya Swain, Co-Founder
Contact	support@devpilot.co.in

Executive Summary

This report presents a comprehensive security audit of the TradePilot trading platform. The audit covers cybersecurity threats, intellectual property exposure, reverse engineering risks, and data protection vulnerabilities.

Key Findings:

- **2 CRITICAL** vulnerabilities requiring immediate action
- **5 HIGH** severity issues affecting IP protection and data security
- **4 MEDIUM** severity issues affecting system integrity

- **4 LOW** severity items for production hardening

The most significant risk is **intellectual property exposure** — the scoring algorithm, ML model weights, and trading strategy are accessible through API endpoints and source code. A competitor with API access could reconstruct the core algorithm within 1-2 weeks. A competitor with source code access could do it in hours.

Threat 1: Telegram Bot Token Exposed

Severity: CRITICAL

What Is It

The TradePilot trading engine sends real-time alerts (buy signals, sell signals, daily profit/loss summaries) to Telegram using a “bot token.” This token is a secret code that proves to Telegram: “I am the TradePilot bot.” This token is stored in a plain text file on the system with no encryption or protection.

Location: `prototype/v5/telegram_config.json`

What Can Go Wrong

If someone obtains this token, they can:

- **Send fake trade alerts** to the Telegram chat. The messages look identical to real alerts. There is no way to tell the difference.
- **Read all past messages** the bot has sent — this includes complete trade history, profit/loss data, position sizes, and stock picks.
- **Impersonate the bot** to anyone in the chat group, including potential clients or partners.
- **Intercept future alerts** — if the bot is used for client notifications later, an attacker could send false buy/sell signals to clients, causing real financial losses.

How It Could Be Exploited

Scenario	Likelihood
Code pushed to GitHub (even private repo with collaborators)	HIGH
Screen visible during a demo or presentation	MEDIUM
Laptop shared or borrowed temporarily	MEDIUM
Computer compromised by malware	LOW

The Solution

1. **Immediately:** Open Telegram, message @BotFather, type /revoke, select the TradePilot bot. BotFather issues a new token. The old token stops working instantly.

2. **Going forward:** Store the token as an environment variable (a value stored in the computer's memory, not in any file). The code reads it at runtime — no file to accidentally share.
 3. **Already done:** The file `telegram_config.json` has been added to `.gitignore` so it will never be committed to version control.
-

Threat 2: User Analytics Database in Version Control

Severity: CRITICAL

What Is It

Every time someone visits the TradePilot website, the system records information about the visit: the visitor's device type, browser, which pages they viewed, which stocks they looked at, what paper trades they made, and feedback they submitted. This data is stored in a file called `tradepilot_analytics.db`.

This file was accidentally committed to the Git repository. Git is a version control system that tracks every change ever made — even if the file is deleted later, Git remembers it forever. It is like a time machine for files.

What Can Go Wrong

Risk	Impact
Repository shared or made public	All visitor data becomes accessible to anyone
Collaborator clones the repository	They receive the database automatically
Legal compliance (Indian IT Act, upcoming DPDPA)	Potential fines and legal liability for exposing personal data

What The Database Contains

- IP addresses of all visitors
- Browser type and device information (iPhone, Chrome, Windows, etc.)
- Complete list of which stocks each visitor searched for or viewed
- All paper trades executed by visitors
- Timestamps showing exactly when each person visited
- Text feedback messages submitted by users

The Solution

1. **Already done:** Added `*.db` files to `.gitignore` — no new database files will be committed
2. **Needs to be done:** Run a Git history cleanup command to permanently remove the file from all past commits

3. **For production:** Move analytics to a proper database server (PostgreSQL) that is never part of the code repository
-

Threat 3: Admin Dashboard Has No Authentication

Severity: HIGH

What Is It

TradePilot has an admin page at the `/admin` URL that displays all analytics — total visitors, active users, page views, stock views, paper trade activity, and user feedback. This is the internal control room for founders.

The problem: there is no password, no login, no security check of any kind. Anyone who types `/admin` after the website URL gets full access.

What Can Go Wrong

- If the site is deployed publicly (it is on Render), anyone on the internet can access `tradepilot.onrender.com/admin`
- A competitor can see exactly how many users the platform has (or does not have)
- Automated bots can scrape all visitor data continuously
- During a client demo, the client could navigate to `/admin` and see raw analytics data

The Solution

Already applied: The `/admin` endpoint now checks if the request comes from `localhost` (the developer's own computer). Requests from the internet receive a "403 Forbidden" error.

For production: Implement a proper login system with username and password protection.

Threat 4: Cross-Origin API Access (CORS Vulnerability)

Severity: HIGH

What Is It

CORS (Cross-Origin Resource Sharing) is a security rule that controls which websites are allowed to call your API. Think of the TradePilot API as a restaurant kitchen, and CORS is the rule about who can place orders.

The original setting was: **any website in the world can order from the kitchen.** This means a competitor's website, a scam website, or a hacker's webpage could silently call TradePilot's API and receive stock scores, trade data, and more.

What Can Go Wrong

Attack	How It Works
Data theft	A competitor builds a website that calls <code>tradepilot.onrender.com/api/scores</code> and displays TradePilot's scores on their own website
Paper trade manipulation	A malicious website calls <code>/api/paper/buy</code> and <code>/api/paper/sell</code> to make trades in the shared paper portfolio
Portfolio destruction	A website calls <code>/api/paper/reset</code> and wipes all paper trading history

The Solution

Already applied: CORS is now restricted to only allow requests from `localhost` (the developer's computer) and `tradepilot.onrender.com` (the official deployment). All other websites are blocked.

Threat 5: Scoring Algorithm Exposed Through API

Severity: CRITICAL (Intellectual Property Risk)

What Is It

This is the single largest intellectual property risk. Several API endpoints return detailed information about how the scoring engine calculates stock scores. This information is sufficient for a competitor to reconstruct the core algorithm.

What Was Being Exposed

The `/api/picks` endpoint returned a `composite_breakdown` field:

This field listed all 7 sub-scores by name (ML Score, Relative Strength, Opening Range Breakout, VWAP, FII Flow, Open Interest, Volume) with their exact numeric values for every stock. This is equivalent to a restaurant publishing the exact recipe for every dish on their menu.

The `/api/scores` endpoint returned detailed reason text:

Instead of generic messages like "Bullish momentum detected," the API returned exact values: "Price +1.43% above VWAP (7059)." This tells a competitor exactly which indicators are used and their precise calculation methodology.

The /api/engine-arena endpoint exposed:

Live positions across all experimental engines, including pool assignments (INTRADAY, SWING, POSITIONAL), direction budgets (50% long / 50% short), regime detection results, and entry/exit prices with stop-loss and target levels.

Reconstruction Risk

Access Level	Time to Rebuild	Effort
API observation only (2 weeks of data)	1-2 weeks	A skilled data scientist
Source code access	1-2 days	Any Python developer
API + paper trade data	3-5 days	Moderate skill required

The Solution

Already applied: API reason text has been sanitized to remove all numeric values. “Price +1.43% above VWAP (7059)” now shows as “Bullish VWAP signal.”

Still needed: Remove the `composite_breakdown` field entirely from API responses. Return only the final score and direction (BUY/HOLD/AVOID), not the individual sub-scores.

Threat 6: ML Model Weights and Architecture Exposed

Severity: HIGH

What Is It

The /api/model endpoint was serving detailed information about the machine learning model:

- **Feature importance rankings:** Which indicators the model considers most important (RSI is number 1, MACD is number 2, etc.)
- **Ensemble weights:** The exact combination of two ML models used (XGBoost at 50.1% and LightGBM at 49.9%)
- **Backtest results:** Historical win rate, Sharpe ratio, total trades, profit factor

Why This Is Dangerous

A competitor reading this endpoint learns: - Exactly which ML models to use (XGBoost + LightGBM) - How to weight them (50/50 split) - Which technical indicators matter most - What performance to benchmark against

This is equivalent to a pharmaceutical company publishing their drug formula on their corporate website.

The Solution

The `/api/model` endpoint should be stripped to only show “Engine: Active” and “Status: Running.” All feature importances, ensemble weights, and backtest metrics should be removed from the public response.

Threat 7: Source Code is a Complete Blueprint

Severity: CRITICAL (if repository becomes public)

What Is It

The Python source code files contain the complete algorithm in readable, commented code. If anyone gains access to the repository, they have a copy-paste-ready blueprint for the entire TradePilot engine.

What Each File Reveals

File	Contents	Damage If Exposed
v4/config.py	All 19 features listed by name, all 7 composite weights with exact percentages, classification thresholds, ORB settings	Complete configuration — can rebuild engine in hours
v4/composite_scorer.py	The full scoring formula, normalization functions, stop-loss and target calculation math	Core algorithm — the “secret sauce”
v5/signal_engine.py	BUY/SELL signal generation rules, percentile rankings, short scoring formula	Signal generation blueprint
v5/regime_detector.py	6-indicator voting system, VIX thresholds, HMM model parameters, allocation multipliers	Market regime classification recipe

File	Contents	Damage If Exposed
v5/pool_manager.py	Capital allocation tables per regime, circuit breaker thresholds, profit waterfall rules	Risk management blueprint

How Fast Can Someone Rebuild?

A skilled Python developer with access to these 5 files can rebuild 90% of TradePilot in a single weekend. The code is well-commented and uses standard Python libraries. No reverse engineering is needed — it is all there in plain text.

The Solution

1. **The GitHub repository must NEVER be made public.** This is the single most important security rule for TradePilot.
2. If sharing code with developers, investors, or auditors, share only frontend code. Never share `prototype/v4/`, `prototype/v5/`, or `scripts/` directories.
3. For long-term protection: move scoring weights to a secure configuration service so they are not stored in code files at all.

Threat 8: Paper Trade Data Reveals Strategy Details

Severity: **HIGH**

What Is It

The `docs/paper-trades/` directory contains JSON files with complete records of every trade the engines have executed. Each trade record includes entry price, stop-loss price, target price, the composite score breakdown, which signals triggered the trade, which pool it was assigned to, and the market regime at the time.

What A Competitor Learns

- Exact stop-loss percentages (approximately 1.5%) and target percentages (approximately 2-3%)
- Which stocks are favored in which market conditions
- Pool assignment patterns (which trades go to SWING versus INTRADAY)
- Win rates and average profit per trade — benchmarks to compete against
- Which specific signals triggered each trade decision

The Solution

Already applied: `docs/paper-trades/` has been added to `.gitignore`. These files will not be pushed to any remote repository.

Threat 9: No User Authentication System

Severity: **HIGH**

What Is It

TradePilot has no login system. There is no concept of user identity. Every API endpoint is open to everyone. The paper trading portfolio is shared by all visitors simultaneously.

What This Means In Practice

- When User A buys RELIANCE in paper trading, User B sees it in their portfolio too
- When User A resets their portfolio, User B's portfolio is also wiped
- A competitor can visit the site, paper trade using the engine, and study its behavior to reverse-engineer the strategy
- During a client demo, the client sees random trades from other visitors

The Solution

Before going live with real money (Kite API integration), a complete authentication system must be built: - User registration and login - Per-user paper trading portfolios stored in a database - API authentication tokens for every request - Role-based access (admin, user, demo)

Estimated effort: 2-3 days of development.

Threat 10: Unpinned Software Dependencies

Severity: **MEDIUM**

What Is It

TradePilot's `requirements.txt` uses version ranges (e.g., `yfinance>=0.2.18`) instead of exact versions (e.g., `yfinance==0.2.18`). The `>=` symbol means "install this version or anything newer."

If a hacker compromises a package and publishes a malicious update, TradePilot's next deployment would automatically install the compromised version.

Real-World Precedent

In 2024, several popular Python packages were hijacked on PyPI (the Python Package Index). Developers using `>=` version pins unknowingly installed malware that stole credentials and API keys.

What Could Happen To TradePilot

- A compromised `yfinance` package could send stock data to a third party
- A compromised `xgboost` could alter ML model predictions, causing bad trades
- A compromised `flask` could intercept all API traffic including trade signals

The Solution

Pin exact versions in `requirements.txt`: use `yfinance==0.2.18` instead of `yfinance>=0.2.18`. Update manually and intentionally, not automatically.

Threat 11: No Input Validation on API Parameters

Severity: MEDIUM

What Is It

When someone visits `/api/stock/RELIANCE`, the server takes “RELIANCE” directly from the URL and passes it to the data library. There is no check on what the user types.

What Could Go Wrong

- A user types an extremely long string (10,000 characters) — could crash the server
- A user types special characters or code fragments — could cause unexpected behavior
- While the current data library handles bad input safely, this is a violation of the security principle of “defense in depth”

The Solution

Add a simple validation rule: only accept 1-20 characters consisting of uppercase letters, numbers, and common symbols (ampersand, hyphen, period).

Threat 12: Server Denial of Service (DDoS)

Severity: MEDIUM

What Is It

The Flask server runs as a single process with 2 workers. If someone sends thousands of requests per second, the server becomes overwhelmed and stops responding to legitimate users.

Why Someone Might Do This

- A competitor wants to disrupt a client demo
- Automated bots attempting to scrape all stock scores rapidly
- Random internet vandalism

The Solution

For production deployment: 1. Place Cloudflare (free tier) in front of the application to block most DDoS attacks 2. Add rate limiting: maximum 30 requests per minute per IP address 3. Increase Gunicorn workers from 2 to 4 or more

Threat 13: Network Interception (Man-in-the-Middle)

Severity: MEDIUM (when deployed publicly)

What Is It

When TradePilot is deployed publicly and users access it over regular HTTP (not HTTPS), anyone on the same network (coffee shop WiFi, office network) can intercept the data flowing between the user's browser and the server. They can see stock scores, paper trades, and all activity in real time.

The Solution

Ensure HTTPS is enabled on all public deployments. Render (the current hosting platform) typically provides HTTPS automatically. Verify that `https://tradepilot.onrender.com` works correctly.

Threat 14: Shared In-Memory State

Severity: MEDIUM

What Is It

The paper trading portfolio is stored in the server's memory (a Python dictionary), not in a database. With Gunicorn running 2 workers, each worker has its own separate copy of the portfolio. Users connected to different workers see different portfolio states.

What This Means

- Trades appear and disappear randomly depending on which worker handles each request

- Portfolio value is inconsistent across page refreshes
- Data is lost every time the server restarts

The Solution

Move paper trading data to a database (PostgreSQL or Redis) so all workers share the same state and data survives restarts.

Summary: Priority Action List

Priority	Threat	Effort	Current Status
DO NOW	Revoke Telegram token at @BotFather	30 seconds	Pending — user action required
DO NOW	Scrub analytics database from Git history	2 minutes	Pending — ready to execute
DONE	Update .gitignore for secrets, trades, models	—	Applied April 15, 2026
DONE	Restrict /admin to localhost only	—	Applied April 15, 2026
DONE	Lock CORS to known origins	—	Applied April 15, 2026
DONE	Sanitize API reason text (strip numeric values)	—	Applied April 15, 2026
NEXT	Remove composite_breakdown from API output	10 minutes	Ready to implement
NEXT	Strip /api/model feature importances	5 minutes	Ready to implement
NEXT	Add API rate limiting	30 minutes	Before public launch
BEFORE LIVE	Build user authentication system	2-3 days	Required for Kite API
BEFORE LIVE	Pin dependency versions	15 minutes	Before next deployment
FOREVER	Never make GitHub repository public	—	Golden rule — permanent

The Golden Rule

The most dangerous thing that can happen to TradePilot is the GitHub repository becoming public.

Every other threat in this report is fixable, recoverable, or mitigable. But once source code is published on the internet, it is there forever. Search engines cache it. Automated scrapers copy it. Competitors download it. There is no “undo” button for a public repository.

The scoring algorithm IS the product. Protect it like the crown jewels.

This report is classified CONFIDENTIAL and intended for internal use only. Do not distribute outside the founding team.

Generated by TradePilot Security Audit System — April 15, 2026